

NesyLink 数理逻辑项目报告

组员： 钱宇航 241880159、吴昊一 241880157、肖思奇 241880615、李沛杰 241880113、高悦雯 231880125

本报告由两个部分组成：第一部分为 Python Agent 策略实现与实验分析，第二部分为 Lean 形式化与证明。两部分共同构成完整的项目提交。

第一部分：Python Agent 策略实现

以下为 Python Agent 部分，重点提供 Agent 策略中适合形式化的抽象层、证明边界、输入约束、策略结构、t1-t5 的实现思路、task 5 的关键调试过程与取舍，以及可视化录像方式。

1. 负责范围与提交文件

Python Agent 主文件：

```
submissions/student_agent.py
```

测评脚本：

```
utils/evaluate_policy.py
```

可视化与录像脚本：

```
utils/watch_agent.py
```

强化学习方向尝试文件：

```
rl_agent/q_learning_agent.py
rl_agent/options_policy.py
rl_agent/train_q_learning.py
rl_agent/train_options_q.py
rl_agent/pretrain_from_expert.py
rl_agent/REPORT.md
```

本 Agent 的目标是：在最终测评阶段只进行推理，不额外训练；推理阶段基于图像帧 **obs**、测评接口显式提供的物品栏信息，以及环境返回的 reward 历史反馈完成决策。当前实现中，策略主要使用 **obs** 经感知模块得到的符号状态，以及物品栏中的钥匙和装备信息；没有直接读取地图真值、房间编号、对象真实坐标、隐藏开关状态或 reward 内部信号。

2. 总体方法

最终采用的是统一的分层符号规划策略，而不是每关单独写死路线：

```
obs 图像帧
-> PerceptionEngine.extract(obs)
-> SymbolicState
-> room_features / room_memory / exit_memory
-> choose_objective
-> BFS / A* / side-directed A*
-> movement_controller / safety shield
-> action
```

其中各层职责如下：

- 感知层：复用已有感知模块，从图像中提取玩家位置、墙、出口、宝箱、按钮、陷阱、桥、怪物等符号信息。
- 特征层：不再判断"这是北房间/东房间/南房间"，而是提取 `room_features`，例如出口集合、是否有桥、是否有按钮、是否有宝箱、是否有怪物、是否有陷阱。
- 记忆层：维护 `room_memory` 与 `exit_memory`，记录已见房间结构、已开宝箱、已按按钮、出口是否通向其他结构房间。
- 目标层：根据当前可见对象、背包钥匙/装备、房间记忆和任务阶段选择目标，例如开宝箱、按按钮、去某侧出口、处理怪物附近的宝箱。
- 规划层：普通目标用 BFS/A*，出口目标用带方向偏置的 A*，减少无意义绕路和频繁转向。
- 安全层：陷阱和墙作为不可走区域；怪物邻近区域加代价；盾牌作为近身兜底，避免明显危险接触。

这种设计比固定路径更适合评分细则中提到的"布局、物体位置或渲染细节略有变化"的测试方式。策略不记忆固定坐标路线，而是根据视觉感知到的对象和出口进行规划。

2.1 可形式化的策略层

评分细则中要求说明"策略形式化与证明"覆盖什么。我们的 Agent 不是端到端神经网络动作策略，而是"视觉感知 + 可验证符号 planner"的组合。因此 Lean 证明不需要证明 CNN 感知模型对所有图像都正确；证明对象应放在感知之后的符号层，也就是假设 `PerceptionEngine.extract(obs)` 给出一个抽象的 `SymbolicState`，再证明 planner 在该状态上满足若干规范。

可形式化层可以拆成以下几个部分：

- 状态抽象：将 Python 中的 `SymbolicState` 抽象为 Lean 中的有限网格状态，包括玩家位置、墙、陷阱、出口、宝箱、怪物、按钮、桥、钥匙数量和已完成交互记忆。
- action mask / 合法动作层：定义 `is_walkable`、`blocked_tiles`、`adjacent_action`、`side_exits` 等谓词，证明 planner 输出的移动动作不会主动走向墙、陷阱、gap、未打开宝箱、怪物占据格或非目标出口。
- 搜索层：普通局部目标使用 BFS/A*，出口目标使用 side-directed A*。Lean 中可证明的核心性质是：若搜索返回路径，则路径首尾合法、相邻步合法、路径上的每个中间格满足可通行条件；若使用 BFS，可进一步证明在有限网格中若存在可行路径则能找到某条可行路径。
- 目标选择层：将策略目标抽象为 `open_chest`、`press_button`、`fight`、`go_exit`、`navigate`、`shield`。证明目标选择只会生成当前状态中存在的目标对象，或在没有局部目标时选择可见出口/可达

frontier。

- 安全层：将怪物邻近、陷阱、出口缓冲区和 shield 规则形式化为 safety predicate。证明在 safety predicate 成立时，planner 不会选择明显危险格；当怪物距离过近且盾牌可用时，策略允许选择 shield 作为安全兜底。
- 轨迹验证层：对执行轨迹定义 progress predicate，例如"宝箱打开后加入 opened set""钥匙数增加后允许尝试 locked exit""进入新房间后更新出口连通关系"。证明如果每一步 action 均合法，且环境执行结果与抽象 transition 一致，那么当轨迹达到 world_completed 对应的目标集合时，任务完成判定成立。

这一划分和代码中的模块对应关系如下：

可验证层	Python 代码位置	Lean 中建议证明的性质
可通行谓词/action mask	<code>blocked_tiles, is_walkable, adjacent_walkable_goals</code>	输出移动目标不在 blocked set 中
局部搜索	<code>bfs_path, astar_path</code>	返回路径相邻、可通行；BFS 在有限图上可证明完备
出口搜索	<code>astar_path_to_side, _go_to_side_via_approach</code>	返回路径朝向目标出口，且不会误入非目标出口
交互动作	<code>_interaction_if_adjacent, _queue_interaction, _advance_interaction</code>	只有相邻且面向目标时执行交互
安全盾牌	<code>_shield_if_close, _emergency_shield_action</code>	危险距离内优先允许 shield，避免直接接触怪物
房间记忆	<code>RoomMemory, ExitMemory, _update_memory</code>	opened/pressed/exit status 单调更新，不反复依赖已完成目标

2.2 证明边界与模型假设

Lean 证明建议采用"符号状态正确性假设"：

若感知层给出的 SymbolicState 与真实画面在可通行格、对象位置和 inventory 上一致，且环境按照动作语义执行，则 planner/action mask/safety shield 约束后的动作满足合法性和安全性规范。

这意味着证明不覆盖 CNN 对任意图像的识别正确性，也不覆盖渲染颜色变化导致的所有感知误差；这些属于感知模型的实验验证范围。证明覆盖的是：一旦视觉模型输出了一个符号状态，后续的目标选择、搜索、action mask、交互和 safety shield 不会随意输出非法动作，并且在可行路径存在且执行成功时能推进到对应目标。

这样的边界符合评分细则中对机器学习/感知模型的要求：模型输出本身不在 Lean 中全称证明，但模型输出进入一个可验证层；该可验证层约束动作合法性、安全性和任务完成谓词。

3. 输入约束与合规性

最终策略的推理过程遵守以下约束：

- 使用 `obs`：策略通过 `PerceptionEngine.extract(obs)` 从图像帧中构造符号状态。
- 使用物品栏：策略读取测评接口显式提供的 `inventory` 中的钥匙数量和装备信息，用于决定是否可开门、是否有剑盾。
- 不使用隐藏状态：策略不读取房间真实编号、地图 JSON、真实对象 id、真实对象坐标、隐藏门状态、怪物内部状态等。
- 不依赖 `reward` 内部信号：调试阶段曾查看 `reward` 事件和地图文件分析失败原因，但当前提交策略不读取 `info["reward"]` 中的内部信号。

需要说明的是，本地调试阶段使用过 `evaluate_policy.py` 输出的事件统计、地图文件和 `reward` 文件来分析失败原因，例如确认 task 5 存在步数/生命压力、确认是否踩陷阱、确认是否打开全部宝箱。这些信息只用于调试和报告分析，不作为最终策略推理输入。

从形式化角度看，输入约束可以理解为一个接口不变量：Python policy 的 `act(obs, info)` 只把 `obs` 交给感知模块，并从 `info` 中读取公开 `inventory` 与上一时刻 `reward` 摘要；它不调用环境对象，不读取地图文件，也不查询真实对象坐标。因此 Lean 侧可以把输入建模为公开观测产生的 `SymbolicState` 加上公开 `inventory`，而不需要把隐藏环境状态加入策略前提。

如果后续使用神经网络感知或训练 objective selector，证明边界仍保持不变：神经网络只提供候选符号状态或候选高层目标；最终动作仍必须经过 action mask、planner 和 safety shield。换言之，模型输出不能直接越过可验证层输出任意动作。

4. 关卡递进分析与策略设计

本项目共有 5 个关卡，难度递增：task 1 和 task 2 是单房间基础机制，task 3 和 task 4 是中等难度的多房间任务链，task 5 是融合多种机制的综合关卡。因此 Agent 不能只写成“每关一条固定路线”，否则虽然可能在公开布局上通关，但难以满足测评中对布局变化和泛化能力的要求。

我们的策略设计原则是：前两关用于验证底层动作是否正确；第三关开始验证跨房间记忆和任务链；第四关引入桥、按钮、剑和怪物后的组合机制；第五关重点考察在生命、步数和多目标约束下能否稳定完成任务。

task 1

task 1 是单房间钥匙与出口任务，目标是取钥匙后离开房间。这一关的机制最简单，但它验证了整个 Agent 最基础的三件事：能否从图像中找到宝箱，能否走到可交互位置并开箱，能否在拿到钥匙后走向出口。

可观察状态包括：

- 玩家 tile 位置与朝向。
- 墙体和可通行区域。
- 宝箱位置。
- 出口位置。
- `inventory` 中的钥匙数量。

策略从视觉中识别宝箱和出口：

1. 若存在未打开宝箱，先移动到宝箱相邻可交互格。
2. 面向宝箱后执行 `ACTION_A` 开箱。
3. 物品栏显示拿到钥匙后，寻找可见出口。
4. 使用统一出口导航 `_go_to_side_via_approach` 离开房间。

这里没有必要引入复杂规划。单房间内只需要 BFS 找到宝箱相邻格和出口 approach tile。这个选择的好处是实现简单、行为可解释，也能作为后续关卡的基础模块。若 task 1 失败，通常说明不是高层规划问题，而是感知、坐标离散化、开箱交互或低层移动控制有问题。

task 2

task 2 仍然是单房间，但加入了怪物机制。相比 task 1，它多验证了攻击动作、怪物阻挡处理和战斗后继续执行任务的能力。

可观察状态包括：

- 怪物位置以及它是否仍被感知为 active。
- 玩家与怪物的相对位置。
- 宝箱和出口位置。
- inventory 中钥匙数量。

策略逻辑为：

1. 若怪物阻挡或需要击杀，则先移动到怪物相邻格，面向怪物后攻击。
2. 怪物被击杀后，开宝箱获得钥匙。
3. 使用统一出口导航到达出口。

这一关的关键不是最短路，而是动作语义：走到怪物附近以后，Agent 不能继续用移动动作顶着怪物走，而应该切换成面向怪物并攻击。调试中还发现，已开宝箱在某些帧可能仍被感知为阻挡物或类似怪物的对象。因此策略维护 `remembered_blockers`，将已交互宝箱加入记忆，避免反复尝试打开或攻击同一位置。

这个设计也为后续关卡提供了一个取舍：怪物处理不应该写成"task 2 专用逻辑"，而应作为通用的 combat/interact 模块。这样 task 4 和 task 5 中出现怪物时，策略可以复用同一套"接近、面向、攻击、确认目标消失"的过程。

task 3

task 3 是多房间任务链，约三房间结构。它的难度来自"当前房间未必直接包含最终目标"，Agent 需要在不同房间之间移动，并根据 inventory 状态决定下一步。

可观察状态包括：

- 当前房间的出口集合。
- 当前房间是否仍有宝箱或怪物。
- inventory 中钥匙数量。
- 已访问过的房间结构签名。
- 已尝试过的出口方向。

策略复用 task 1/2 的基础动作：

1. 在当前房间找可交互目标。
2. 若当前房间目标已完成，则根据可见出口前往下一房间。
3. 物品栏拿到钥匙后返回可开锁出口。

task 3 说明仅靠"当前房间贪心"已经不够。Agent 需要知道自己曾经从哪个出口进入过其他房间，避免在两个房间之间无意义来回。因此我们加入了轻量记忆机制。这里的 memory 不是环境真值房间编号，而是由视觉结构

生成的房间签名；在普通 hub 探索中，签名主要来自出口方向、桥方向、switch/button/monster 等可见特征；在更复杂的全局 planner 中，签名进一步使用出口集合和墙体结构，避免把房间真实 id 作为策略输入。

这一关的策略仍然不是全局最优规划，但它建立了后续 task 4 和 task 5 所需的跨房间状态：哪些房间见过、哪些出口试过、哪些房间仍有可交互对象。

task 4

task 4 是五房间结构，中央桥接房需要按按钮切换连通方向。玩家初始无剑，需要依次完成"取钥匙 -> 开宝箱取剑 -> 击杀小怪 -> 开启胜利宝箱"的任务链条。它比 task 3 更难的地方在于：房间之间并不是静态全连通，中央桥的状态会改变可达出口。

可观察状态包括：

- 当前房间出口集合。
- 是否存在桥、按钮、宝箱、怪物、陷阱。
- 桥 tile 与房间边界的接触关系。
- inventory 中钥匙和剑的状态。
- 已访问房间和已打开宝箱。

早期实现容易写成"去北边拿钥匙、去东边拿剑、去南边杀怪"这种固定方向策略，但这不利于泛化。最终实现改成基于事实的 `room_features`：

```
exits
has_bridge
has_switch
has_button
has_chest
has_monster
has_trap
bridge_sides
```

桥方向也不再用"北/南/东房间"判断，而是通过桥 tile 接触的边界推断可连接的 sides。例如桥 tile 接触房间上边界，就说明桥可能连接 `up`；接触右边界，就说明可能连接 `right`。策略只关心"当前桥连接哪些出口"和"当前房间是否有关键对象"，不关心房间名字。

task 4 的高层逻辑为：

1. 在桥接/中心结构中寻找可达出口。
2. 若桥未连到目标出口，则找 switch/button 改变连通。
3. 找包含宝箱的房间，开箱获取钥匙或装备。
4. 拥有剑后处理怪物。
5. 怪物目标完成后寻找最终宝箱。

这里的核心分析是：task 4 的任务顺序看起来固定，但真正固定的是"依赖关系"，不是"房间方向"。钥匙必须在开锁宝箱前获得，剑必须在击杀怪物前获得，怪物清理后才能安全接近最终宝箱。因此策略写成需求驱动：

- 没有 key 时，寻找当前可达的未开宝箱。
- 有 key 但没有 sword 时，探索可达出口并寻找宝箱。

- 有 sword 后，处理怪物或怪物附近目标。
- 怪物清空后，寻找最终宝箱或剩余交互点。

移动、开箱、战斗和出口导航都复用通用函数，task 4 只是在目标选择上增加桥房间探索逻辑。这个取舍比单独写 `_act_task4` 更有利于满足"统一策略覆盖多关卡"的评分点。

当前代码中，task 4 主要走 local/hub planner，而不是强制开启完整全局搜索。进入 hub 模式的触发条件是看见 switch 与墙体结构，或看见 bridge 与 gap；之后策略维护 `switch_hub_side`、`current_target_side`、`explored_hub_sides`、`hub_scanned_frontiers` 等状态。这样做的目的不是记住"某个固定方向是什么房间"，而是记住"当前桥或 switch 正在连接哪些 side，以及哪些 side 已经探索过"。桥约束也不是简单地"看见 bridge 就只能走 bridge"，而是由 `bridge_requires_constrained_walk` 判断：只有 bridge 邻接 gap，或 bridge 连接两个及以上出口方向时，才把它视为必须受限通行的桥结构。

task 5

task 5 是综合关卡，融合多房间、钥匙门、治疗宝箱、金币宝箱、按钮、怪物、陷阱和步数/生命压力。公开环境中的目标是通关，本地事件显示关键完成条件是打开所有关键宝箱并使 `world_completed=True`。这一关不是简单地"能不能找到目标"，而是"能不能在有限生命和有限步数内以足够短的路径完成多个目标"。

4.5.1 可达性分析

task 5 的可达性依赖以下任务链：

1. 从起点房间获得初始可交互收益或触发必要按钮。
2. 探索普通出口，进入包含 key chest 的房间。
3. 拿到 key 后回到起点附近。
4. 进入需要 key 的出口，打开后续宝箱。
5. 处理治疗宝箱、金币宝箱和可能挡路的怪物。
6. 在生命耗尽前打开全部关键宝箱。

这说明 task 5 的失败原因可能有三类：

- 目标顺序错误：例如没有钥匙却反复尝试钥匙门。
- 路径效率低：目标顺序正确，但绕路导致步数或生命不够。
- 安全控制差：被怪物或陷阱消耗过多生命。

调试后发现，我们的早期版本主要不是第一类问题。Agent 拿到钥匙后，高层目标确实选择了右侧钥匙门，但局部路径规划为了避开墙、npc、怪物邻接格，会先向上走几步再折返向右。这个行为单次看只浪费几十 tick，但 task 5 的生命和步数都很紧，最后会导致宝箱来不及打开。

4.5.2 关键状态可观察性

task 5 中有些状态可以直接从视觉和 inventory 观察，有些只能通过交互后的变化间接确认：

- 可直接观察：玩家位置、墙、出口、陷阱、怪物、宝箱、按钮、桥/通路结构。
- 可由 inventory 观察：钥匙数量、剑/盾等装备状态。
- 可由记忆确认：某个宝箱是否已经开过，某个出口是否已经尝试过，某个房间是否仍有未处理目标。
- 不直接依赖：地图真值、房间真实编号、隐藏对象坐标、环境内部 `info`。

这里的关键取舍是：宝箱内容不一定能在打开前从图像稳定判断，所以策略不预设"某个方向一定是治疗宝箱/金币宝箱/钥匙宝箱"。它只维护"看见宝箱 -> 尽量打开 -> 根据 inventory 或完成状态更新记忆"的闭环。这比固定写"先去某个房间拿某个物品"更符合最终测评要求。

最终 task 5 的策略包括：

1. 全局房间记忆：`RoomMemory` 记录已知出口、已知宝箱、已开宝箱、已按按钮、是否见过怪物/按钮、访问过的 inventory revision；`ExitMemory` 记录某个结构房间某侧出口的状态、目标房间和出口类型。
2. 需求驱动目标：优先处理当前房间未开宝箱或按钮；当前房间完成后，选择仍有未完成目标或未知 frontier 的出口。
3. 出口状态判断：出口类型分为 `normal`、`locked_key`、`conditional`、`unknown`。若出口被挡但钥匙或条件后来满足，允许 retry；若出口已知通向仍有未完成目标的房间，则优先返回。
4. 加权 A*/BFS：墙、陷阱、gap、未打开宝箱、怪物和非目标出口作为 blocked 或高风险区域；普通目标用 BFS/A*，出口目标用 side-directed A*。
5. side-directed A*：当目标是"去某一侧出口"时，路径规划额外惩罚反方向移动、无意义垂直/水平绕路和频繁转向，并优先选择离当前行/列最近的 approach tile。
6. safety shield 与低血量出口保护：怪物距离过近时可以举盾；低血量进入未知或未确认 open 的出口前，会先执行一次 shield，避免切房或门口接触时被扣血。
7. stuck recovery：若连续移动后玩家 tile 没有变化，或上一动作被 blocked reward 反馈确认，策略会记录 blocker、清空当前 commit，并尝试垂直/水平恢复动作，避免无限撞墙。

task 5 最关键的改动是把"去某侧出口"做成可复用的出口 planner，而不是硬编码"回起点后直接向右"。当前 `_go_to_side_via_approach` 的逻辑是：

1. 根据目标 side 找到该侧所有出口 tile。
2. 将非目标出口临时加入 blocked set，避免路径规划误入其他房间。
3. 计算目标出口的 approach tile，例如上出口对应 `(x, 1)`，左出口对应 `(1, y)`。
4. 若已在 approach tile 上，先做 lane alignment，再向目标出口移动。
5. 若尚未到 approach tile，用 side-directed A* 规划路径。
6. 若低血量且当前房间有多个出口、没有剩余宝箱，则额外检查普通路径是否会很早触发"靠近怪物需要盾牌"的情况；必要时重新运行带 preventive shield 成本的 A*，优先选择更安全的出口路线。

side-directed A* 的通用路径代价包括：

- 朝目标 side 前进的动作代价略低。
- 远离目标 side 的动作代价较高。
- 垂直/水平绕行动作有小惩罚。
- 频繁转向有额外惩罚。
- approach tile 优先选择离当前行/列最近的候选。

当前版本还加入了两类和"执行稳定性"相关的机制：

- 时间滤波与交互确认：策略通过 `TemporalSymbolicFilter` 平滑感知到的符号状态，并在开箱、按按钮等交互后等待 inventory/reward 或目标消失来确认交互完成；已打开宝箱会进入 memory/blocker，避免反复处理同一位置。
- 像素中心微调：离散 tile 决策之外，低层 controller 会用 `player_entity.center_px` 做短步居中修正。这样 A*/BFS 返回的是 tile 路径，而具体动作执行时仍能处理玩家站在 tile 边缘、出口 lane 未对齐等问题。

这些机制都不是针对某一关写固定坐标。它们的共同点是只依赖当前视觉抽象、inventory、上一时刻 reward 和策略内部记忆。

4.5.3 鲁棒性分析

这个方案的鲁棒性来自以下层面：

- 对房间方向的鲁棒性：策略不把"下方房间是 key 房、右方房间是治疗房"写死，而是根据出口、宝箱和 inventory 状态动态选择目标。
- 对局部布局的鲁棒性：路径由 A*/BFS 根据当前可通行区域计算，不依赖固定像素坐标序列。
- 对危险对象的鲁棒性：墙、陷阱和 gap 不可走，怪物和怪物邻近格提高代价；近距离危险时 safety shield 可以兜底。
- 对出口探索的鲁棒性：出口有 unknown/open/blocked 状态；locked exit 在拿到 key 后可以重试，conditional exit 在按钮/怪物/交互完成后可以重试。
- 对短暂感知抖动的鲁棒性：时间滤波、交互确认、已开宝箱记忆和 blocked recovery 减少"同一个对象被重复处理"或"卡在墙边持续撞墙"的情况。

它的局限也很明确：

- 如果视觉感知把墙、出口、宝箱或怪物持续识别错，规划层仍会基于错误符号状态决策；当前修复主要处理短暂抖动和动态对象导致的记忆不一致，不能替代感知模型本身的准确率。
- 当前 memory 是轻量房间图和启发式出口选择，不是完整资源约束最优规划；如果测试关卡更长、更随机，可能需要把 (room_signature, opened_chests, keys, hp_estimate, exit_status) 纳入全局 Dijkstra/A* 状态。
- 目前没有训练一个 learned objective selector，因此目标选择仍然主要依赖人工设计的规则优先级。

这一版策略说明的重点是可解释、可验证和可迁移的策略结构；具体通关数量、变体覆盖率和中间目标统计由后续实验结果部分统一报告。

5. 方法选择与 RL Agent 尝试

实现过程中参考过类似 Zelda 游戏的 RL 仓库，也考虑过 PPO、tabular Q-learning 和分层 option-Q 方案。最终提交主线没有采用端到端纯 RL，而是选择"规则目标选择 + 搜索规划 + 记忆图 + safety shield"的混合符号方法；但项目中保留了一个独立的 rl_agent 方向，用来验证学习式高层策略是否能够替代一部分人工目标优先级。

5.1 为什么最终主线不是端到端纯 RL

端到端动作 RL 在本任务中有几个明显风险：

- 训练样本效率不确定，课程时间内难保证 task 1 到 task 5 都稳定通关。
- 纯 RL 很难解释为什么泛化，也难以和 Lean 中可证明的 planner/action mask/safety shield 对齐。
- 参考仓库中的 RL 环境读取了模拟器内存地址，例如房间号、血量、位置、钥匙状态。这类信息在本任务最终测评中不能直接依赖。
- 本项目已有视觉感知和 t1-t4 规则+BFS 基础，继续升级为统一符号 planner 的收益更高，也更容易定位 task 5 的失败原因。

因此，正式提交策略优先保证可解释性、可复现性和与最终测评接口的一致性。RL 没有作为最终兜底路线接入 submissions/student_agent.py，而是作为单独实验目录保留。

5.2 RL Agent 的两阶段尝试

第一阶段实现了普通 tabular Q-learning :

```
rl_agent/train_q_learning.py
rl_agent/features.py
rl_agent/models/q_table.pkl
```

这一路线使用 `observation_mode="grid"` 的结构化观测训练 $Q(s, a)$ ，状态键包含任务、房间、玩家位置、血量、钥匙数、剩余怪物/宝箱/出口和归一化 grid。它适合验证基础学习流程和模型保存/加载，但动作空间仍是逐 tick 的底层动作。对于 task 4/5 这种跨房间、长 horizon、稀疏奖励任务，普通表格 Q-learning 很难在有限 episode 内学到稳定长程策略。

第二阶段改成分层 options RL :

```
rl_agent/q_learning_agent.py
rl_agent/options_policy.py
rl_agent/train_options_q.py
rl_agent/models/options_q.pkl
```

默认入口 `rl_agent.q_learning_agent` 指向 `OptionsRLPolicy`，不在线导入 `submissions/student_agent.py`。高层动作不再是单步上下左右，而是通用 option :

```
open_chest / fight_monster / press_button / press_switch /
go_exit(up/right/down/left)
```

底层仍由 option controller、BFS/局部移动、交互控制和阻塞记忆执行。这样做的目的不是复制教师轨迹，而是让 Q-learning 学习"当前状态下应该先开箱、按按钮、战斗还是探索哪个出口"。`train_options_q.py` 默认已经把 task 1 到 task 5 都纳入训练集合：

```
python -m rl_agent.train_options_q --episodes 400
```

5.3 RL Agent 当前效果

根据 `rl_agent/REPORT.md` 中记录的阶段结论，RL agent 已经完成以下工作：

- 默认入口独立于 `submissions/student_agent.py`，避免评测时依赖最终符号 agent 兜底。
- `train_options_q.py` 默认训练 task 1 到 task 5，而不是只在简单关卡训练。
- `options_policy.py` 使用通用 option 空间，包括开箱、战斗、按钮/开关和出口探索。
- 加入跨房间打断、已阻塞格记忆、怪物危险区避让等通用机制，避免写入 task 5 专属房间路线。

当前阶段的结论是：task 1 到 task 4 此前已经可以通过；task 5 已纳入训练与验证范围，但 RL agent 还没有稳定通关。它在 task 5 中能够稳定完成起点房间开箱拿金币、按按钮打开南侧条件出口、进入南房间打开钥匙

箱、返回起点后打开东侧锁门、进入东房间打开治疗箱。主要瓶颈是从东房间返回以及继续探索西房间时，出口定位和持续移动会消耗大量步数；task 5 又有生命随时间衰减的压力，导致到达最后目标前血量偏低。

5.4 不过拟合与取舍

RL agent 的尝试刻意避免三类过拟合：

- 不写 `task_5` 专属房间名、固定坐标路线或"从某房间走到某房间"的硬编码脚本。
- 不在线调用 `student_agent` 作为 teacher 或 fallback；`pretrain_from_expert.py` 只作为离线实验脚本保留，默认 agent 不导入。
- 高层 option 基于局部可见状态、库存、房间访问记忆和阻塞记忆选择，而不是读取地图 JSON 或环境隐藏真值。

这说明 RL 方向的定位更像"学习式 objective selector"。它和最终符号 planner 并不冲突：比较合理的后续方案是让 RL 只选择 `open_chest` / `press_button` / `go_exit` / `fight` / `return` 等高层目标，底层动作仍由 planner 和 safety shield 约束。这样即使 learned selector 偶尔选择不理想目标，也不会直接输出危险动作。

当前没有把 RL agent 并入最终提交主线，主要原因是 task 5 尚未稳定通关，而符号 planner 已经能在公开环境和 robustness suite 中稳定完成 t1-t5。后续若继续推进，优先改进方向包括：为 `train_options_q.py` 增加开箱、开门、治疗、换房等稀疏正奖励 shaping；在 option 状态中加入健康分桶和剩余宝箱记忆；修正玩家站上出口 tile 后出口从可见集合中消失导致的边界振荡；最后分别评测加载 Q 表和禁用 Q 表两种模式，确认收益确实来自高层 option-Q，而不是手工规则。

6. 调试尝试与取舍

实现过程中做过几类尝试：

- 参数搜索：尝试调整怪物风险、陷阱风险、未知出口奖励、blocked 出口惩罚等参数，但单纯调权重不能解决 task 5 的关键浪费。
- 更强 combat controller：尝试将怪物处理改成"攻击优先、盾牌兜底、怪物挡路才战斗"。这对解释性有帮助，但 task 5 的主要瓶颈不是怪物战斗，而是出口路径绕行。
- 全局多目标 planner：最终版本保留了一个轻量全局 planner，只在看见 button 后启用。它用 `RoomMemory/ExitMemory` 记录出口连通、已开宝箱、已按按钮、未知 frontier 和 blocked exit；但没有使用完整资源状态空间搜索，而是用启发式 priority + path length score 选择下一出口。这样可以解释出口选择，也避免全局状态爆炸。
- 感知接口更新后的兼容调试：同步新版 perception 后，t4/t5 一度不能通过。诊断发现，失败不是高层目标完全错误，而是短暂感知抖动、开箱后对象残留、桥/通路语义过强、出口处像素未对齐等因素共同导致。最终保留的修复是时间滤波、已开宝箱记忆、桥约束语义化、像素级 movement/exit alignment、blocked recovery，以及低血量出口前 shield。
- 过度清理尝试：我们也尝试删除部分看似特殊的分支，例如低血量出口避怪二次路径、普通移动像素对齐等。回归验证显示这些机制会影响 spatial 变体或 task 5，因此最终保留。真正删掉的是对低血量 safe chest 的单独 path config 覆盖，因为它不再影响当前策略通过情况。

最终保留的是对泛化和证明最有帮助的机制：统一出口导航、房间特征、轻量记忆图、需求驱动目标、side-directed A*、action mask、stuck recovery 和 safety shield。没有保留的是每关固定路线、固定坐标和直接读取环境内部状态。

7. 可视化与真实性材料

为满足"能证明代码确实运行"的提交要求，补充了 Agent 可视化脚本：

```
utils/watch_agent.py
```

实时观看某一关：

```
.venv/bin/python utils/watch_agent.py \  
--task mathematical_logic/task_5 \  
--policy submissions/student_agent.py \  
--seed 0 \  
--max-steps 1400 \  
--fps 30
```

生成 t1-t5 录像：

```
mkdir -p outputs/agent_videos  
for i in 1 2 3 4 5; do  
    .venv/bin/python utils/watch_agent.py \  
    --task mathematical_logic/task_${i} \  
    --policy submissions/student_agent.py \  
    --seed 0 \  
    --max-steps 1400 \  
    --no-window \  
    --video-out outputs/agent_videos/task_${i}.mp4 \  
    --video-fps 30  
done
```

若缺少视频依赖：

```
.venv/bin/python -m pip install "imageio[ffmpeg]>=2.34"
```

macOS 上如果出现 SSL 证书问题，可临时使用：

```
.venv/bin/python -m pip install \  
--trusted-host pypi.org \  
--trusted-host files.pythonhosted.org \  
"imageio[ffmpeg]>=2.34"
```

实时窗口快捷键：

- **Space**：暂停/继续
- **N**：单步执行
- **R**：重置当前 episode

- `Esc` : 退出

该可视化脚本不改变策略行为，只用于观察与保存运行过程。

8. 正式实验报告

8.1 测评环境与代码版本

项目	内容
Policy 文件	<code>submissions/student_agent.py</code>
辅助模块	<code>submissions/temporal_filter.py</code> （时序符号滤波，抑制感知短期抖动）
感知模型权重	<code>nesylink/perception/perception_model.pt</code>
RL 模型权重（可选）	<code>rl_agent/models/q_table.pkl</code> 、 <code>rl_agent/models/options_q.pkl</code>
测评脚本	<code>utils/evaluate_policy.py</code>
代码版本（Git commit）	<code>6d8d86a</code>
Python 版本	<code>>=3.10</code>
环境配置	<code>observation_mode="pixels", action_repeat=1</code> （未覆盖）

8.2 任务默认配置（未覆盖）

各任务在 `nesylink/tasks/task_config/mathematical_logic.yaml` 中的默认配置如下，正式测评未使用 `--max-steps` 或 `--action-repeat` 覆盖：

task	max_steps	action_repeat
<code>mathematical_logic/task_1</code>	500	1
<code>mathematical_logic/task_2</code>	500	1
<code>mathematical_logic/task_3</code>	1500	1
<code>mathematical_logic/task_4</code>	2000	1
<code>mathematical_logic/task_5</code>	2000	1

8.3 正式测评命令

正式成绩以 `--info-mode safe --robustness-suite` 的实际运行结果为准：

```
.venv/bin/python utils/evaluate_policy.py \  
  --policy submissions/student_agent.py \  
  --tasks mathematical_logic/task_1 mathematical_logic/task_2 \  
         mathematical_logic/task_3 mathematical_logic/task_4 \  
         mathematical_logic/task_5 \  

```

```
--info-mode safe \  
--robustness-suite \  
--num-envs 100 \  
--seed 0 \  
--json-out outputs/robustness_suite_eval.json
```

参数说明：

- `--info-mode safe`：策略只收到 `last_reward` 和 `inventory`，不接收环境内部状态。
- `--robustness-suite`：启用固定比例鲁棒性套件（60% original + 30% spatial + 10% color）。
- `--num-envs 100`：每个 task 共 100 个 episode，按 60/30/10 比例分配。
- `--seed 0`：episode seed 为 `0 + episode_index`。
- 未传递 `--max-steps` 和 `--action-repeat`，使用任务默认配置。

8.4 测评结果

8.4.1 各阶段成功率

task	original (60ep)	spatial (30ep)	color (10ep)
mathematical_logic/task_1	100.0%	100.0%	100.0%
mathematical_logic/task_2	100.0%	100.0%	100.0%
mathematical_logic/task_3	100.0%	100.0%	100.0%
mathematical_logic/task_4	100.0%	100.0%	100.0%
mathematical_logic/task_5	100.0%	100.0%	100.0%

8.4.2 完整指标汇总

以下为 `--robustness-suite --num-envs 100 --seed 0` 的实际运行结果：

task	stage	episodes	success_rate	avg_steps	avg_reward
mathematical_logic/task_1	original	60	100.0%	283.0	127.170
mathematical_logic/task_1	spatial	30	100.0%	178.0	128.170
mathematical_logic/task_1	color	10	100.0%	283.0	127.170
mathematical_logic/task_2	original	60	100.0%	182.0	126.180
mathematical_logic/task_2	spatial	30	100.0%	183.7	126.563
mathematical_logic/task_2	color	10	100.0%	182.0	126.180
mathematical_logic/task_3	original	60	100.0%	545.0	164.550
mathematical_logic/task_3	spatial	30	100.0%	648.7	163.497
mathematical_logic/task_3	color	10	100.0%	545.0	164.550

task	stage	episodes	success_rate	avg_steps	avg_reward
mathematical_logic/task_4	original	60	100.0%	1085.0	249.150
mathematical_logic/task_4	spatial	30	100.0%	1374.0	263.227
mathematical_logic/task_4	color	10	100.0%	1064.4	250.156
mathematical_logic/task_5	original	60	100.0%	1188.0	145.970
mathematical_logic/task_5	spatial	30	100.0%	1163.0	146.920
mathematical_logic/task_5	color	10	100.0%	1189.0	145.950

8.4.3 Milestone 达成率

task	stage	milestone	rate
mathematical_logic/task_3	original	monster_killed	100.0%
mathematical_logic/task_3	original	key_collected	100.0%
mathematical_logic/task_3	spatial	monster_killed	100.0%
mathematical_logic/task_3	spatial	key_collected	100.0%
mathematical_logic/task_3	color	monster_killed	100.0%
mathematical_logic/task_3	color	key_collected	100.0%
mathematical_logic/task_4	original	switch_activated	100.0%
mathematical_logic/task_4	original	key_collected	100.0%
mathematical_logic/task_4	original	door_opened	100.0%
mathematical_logic/task_4	original	item_collected	100.0%
mathematical_logic/task_4	original	monster_killed	100.0%
mathematical_logic/task_4	spatial	switch_activated	100.0%
mathematical_logic/task_4	spatial	key_collected	100.0%
mathematical_logic/task_4	spatial	door_opened	100.0%
mathematical_logic/task_4	spatial	item_collected	100.0%
mathematical_logic/task_4	spatial	monster_killed	100.0%
mathematical_logic/task_4	color	switch_activated	100.0%
mathematical_logic/task_4	color	key_collected	100.0%
mathematical_logic/task_4	color	door_opened	100.0%
mathematical_logic/task_4	color	item_collected	100.0%
mathematical_logic/task_4	color	monster_killed	100.0%

8.4.4 Progress 指标

以下为各 task 在各阶段至少出现一次的 progress 事件比例（仅列出实际出现的事件）。

task 1（全阶段一致）：

progress 事件	original	spatial	color
chest_opened	100.0%	100.0%	100.0%
door_opened	100.0%	100.0%	100.0%
environment_completed	100.0%	100.0%	100.0%
exit_reached	100.0%	100.0%	100.0%
key_collected	100.0%	100.0%	100.0%
room_changed	100.0%	100.0%	100.0%
world_completed	100.0%	100.0%	100.0%

task 2（全阶段一致）：

progress 事件	original	spatial	color
chest_opened	100.0%	100.0%	100.0%
environment_completed	100.0%	100.0%	100.0%
exit_reached	100.0%	100.0%	100.0%
key_collected	100.0%	100.0%	100.0%
monster_killed	100.0%	100.0%	100.0%
room_changed	100.0%	100.0%	100.0%
world_completed	100.0%	100.0%	100.0%

task 3（全阶段一致）：

progress 事件	original	spatial	color
chest_opened	100.0%	100.0%	100.0%
door_opened	100.0%	100.0%	100.0%
environment_completed	100.0%	100.0%	100.0%
exit_reached	100.0%	100.0%	100.0%
key_collected	100.0%	100.0%	100.0%
monster_killed	100.0%	100.0%	100.0%
room_changed	100.0%	100.0%	100.0%

progress 事件	original	spatial	color
world_completed	100.0%	100.0%	100.0%

task 4（全阶段一致）：

progress 事件	original	spatial	color
chest_opened	100.0%	100.0%	100.0%
door_opened	100.0%	100.0%	100.0%
environment_completed	100.0%	100.0%	100.0%
exit_reached	100.0%	100.0%	100.0%
gold_collected	100.0%	100.0%	100.0%
item_collected	100.0%	100.0%	100.0%
key_collected	100.0%	100.0%	100.0%
monster_killed	100.0%	100.0%	100.0%
room_changed	100.0%	100.0%	100.0%
world_completed	100.0%	100.0%	100.0%

task 5（全阶段一致）：

progress 事件	original	spatial	color
agent_healed	100.0%	100.0%	100.0%
button_pressed	100.0%	100.0%	100.0%
chest_opened	100.0%	100.0%	100.0%
door_opened	100.0%	100.0%	100.0%
environment_completed	100.0%	100.0%	100.0%
exit_reached	100.0%	100.0%	100.0%
gold_collected	100.0%	100.0%	100.0%
key_collected	100.0%	100.0%	100.0%
room_changed	100.0%	100.0%	100.0%
world_completed	100.0%	100.0%	100.0%

task 5 的 `item_collected`、`monster_killed`、`trap_triggered` 在所有 episode 中的 milestone 达成率均为 0.0%，说明策略在 task 5 中不依赖击杀怪物、收集物品或触发陷阱来完成目标，而是通过开宝箱、按按钮、开门和换房的方式达成 `world_completed`。

8.5 本地单 seed 验证结果

以下为 `--seed 0 --num-envs 1` 在原始地图上的单次验证结果，仅用于说明策略在当前公开环境下的基线表现：

任务	是否通关	步数	reward	说明
task 1	是	283	127.170	开箱取钥匙并出门
task 2	是	182	126.180	击杀怪物、取钥匙并出门
task 3	是	545	164.550	多房间钥匙链完成
task 4	是	1085	249.150	桥/按钮/钥匙/剑/怪物/最终宝箱完成
task 5	是	1188	145.970	4 个宝箱全部打开， <code>world_completed=True</code>

task 5 seed=0 事件统计：

```
chest_opened=4, key_collected=1, gold_collected=2,
agent_healed=1, button_pressed=1, door_opened=1,
room_changed=5, exit_reached=5,
action_shield=10, shield_block=1, world_completed=1
```

task 5 的 agent 在 seed=0 中进行了 10 次举盾、1 次成功格挡，4 次打开宝箱（含钥匙宝箱、治疗宝箱和金币宝箱），1 次按按钮、1 次开门，共换房 5 次，最终触发 `world_completed`。

8.6 训练/调试阶段 info 使用说明

训练和调试阶段使用过完整环境 `info (--info-mode full)`，用途如下：

- **感知模型训练**：`nesylink/perception/cnn.py` 的 `collect` 子命令使用 `observation_mode="full"` 获取 structured observation 作为 CNN 训练标签，包括 tile 语义图、出口类型、宝箱开闭状态、玩家/怪物 heatmap。这些标签只用于离线训练感知模型权重，不进入策略推理流程。
- **调试与问题排查**：使用 `evaluate_policy.py --info-mode full` 输出的事件统计（`info["events"]`）、reward 细分信号（`info["reward"]`）和终端原因（`info["terminal_reason"]`）分析 task 5 步数/生命压力、确认陷阱触发情况和宝箱完成状态。
- **地图结构分析**：查看 `info["env"]["room_id"]` 和 `info["dynamic"]` 理解桥连通逻辑和房间空间关系，用于设计 `room_features` 和桥约束规则。

以上 `info` 使用不进入最终策略推理阶段。正式测评使用 `--info-mode safe`，策略仅接收 `last_reward` 和 `inventory`。

9. 局限性与后续可改进点

当前策略仍是符号规则与搜索方法，不是端到端学习方法。它的优势是可解释、可调试、容易和 Lean 中的安全约束或 planner 层证明对应；局限是依赖感知模块输出质量。如果渲染风格或对象颜色变化较大，首先需要保证感知模块仍能稳定提取符号状态。

另外，当前实现保留的是轻量房间记忆图，而不是严格的资源约束最优规划。这样做是有意取舍：公开 task 5 的主要失败点是局部路径绕行和危险规避，而不是高层目标完全未知；轻量方案能覆盖当前 t1-t5，并避免全局

状态搜索带来的状态膨胀、估计误差和调参不稳定。若未来测试关卡更长、更随机，可以进一步扩展为以 `(room_signature, opened_chests, keys, hp_estimate)` 为状态的 Dijkstra/A* 多目标 planner；现有的宝箱记忆、钥匙判断、出口探索和 safety shield 都可以作为该 planner 的状态变量与动作约束。

第二部分：Lean 形式化与证明

以下为完成的 Lean 形式化与证明工作。第一部分中讨论的"可形式化策略层"与"证明边界"在此处得到了具体的 Lean 实现。两部分的分工对应关系为：第一部分第 2.1 节列出的可验证层（可通行谓词、局部搜索、出口搜索、交互动作、安全盾牌、房间记忆）在本部分中被抽象为 `Strategy.lean` 中的六层架构，并基于公理体系证明了安全性与活性定理。Lean 部分的核心代码文件包括 `Env1.lean`（基础类型）、`Env2.lean`（状态转移）、`Strategy.lean`（策略建模）、`StrategyProof.lean`（定理与证明）和 `Axioms.lean`（公理体系），均位于 `lean/NesyLink/` 文件夹中。

1. 环境形式化

简化假设

为了便于形式化和后续证明，我们对游戏模型进行了以下假设。

网格移动

玩家和怪物只能在格子的中心移动。为了建模怪物的速度只有玩家的一半，我们设计每回合玩家行动一次（向一个方向移动一格，向一格方向攻击/交互，防御），每两回合怪物行动一次。

怪物模型简化

怪物类型简化，假设所有的怪物都是 chaser，向玩家移动。怪物靠近玩家造成伤害后反弹，被玩家攻击后也会反弹，反弹之后忽略怪物晕眩的时间。由于只有个别关卡的个别房间有两个怪物，忽略怪物重叠的情况。

游戏砖块简化

策略中并没有针对 npc 的逻辑，我们考虑将 npc 视作普通墙体。陷阱类型简化，假设所有的陷阱都是 spike，玩家踩到后扣血并回到当前房间重生点。

基础类型（`Env1.lean`）

坐标：

环境中的地图房间固定为 8 行 × 10 列的网格，每个格子由一个坐标唯一标识。在 Lean 中，我们用有限类型 `Fin` 来建模坐标：

```
def Coord := Fin 8 × Fin 10
```

这里 `Fin 8` 表示 $\{0, 1, \dots, 7\}$ ，`Fin 10` 表示 $\{0, 1, \dots, 9\}$ 。使用 `Fin` 而非 `Nat` 的原因如下：`Nat` 无法在类型层面排除越界坐标；每次访问地图都要携带 $h : \text{row} < 8 \wedge \text{col} < 10$ 的证明；而 `Fin 8` 和 `Fin 10` 类型本身保证了坐标合法——所有类型正确的 `Coord` 值都自动在 $0 \dots 7 \times 0 \dots 9$ 范围内。

方向：

游戏是二维的，因而只有上下左右四个方向，且每个时刻只能有一个方向。在 Lean 中，我们用一个枚举类型来建模方向：

```
inductive Direction where
  | up | down | left | right
```

动作：

游戏中只有四种动作：等待，移动（4个方向），攻击或交互（buttonA），防御（buttonB）。在 Lean 中，我们用一个枚举类型来建模动作：

```
inductive Action where
  | wait | move | buttonA | buttonB
```

输入结构：

每一个时刻的输入由两个部分组成：动作和方向。我们规定：如果动作是移动，那么方向是玩家将要移动的方向；如果动作是另外三个，那么方向是交互的方向。在 Lean 中，我们用一个结构体来建模输入：

```
structure Input where
  direction : Direction
  action : Action
```

地图格子：

在游戏中，一个地块有八种可能（地面、墙、陷阱、按钮、开关、宝箱、桥、门）。在 Lean 中，我们用一个枚举类型来建模地块：

```
inductive Tile where
  | ground
  | wall
  | spike
  | button (pressed : Bool)
  | switch (state : Nat)
  | chest (opened : Bool) (content : Item)
    (hidden : Bool) (cond : Condition)
  | bridge (switchRoom : Nat)
    (switchCoord : Coord) (activeState : Nat)
  | door (id : Nat)
```

特别解释一下bridge的三个参数：switchRoom是控制当前桥的开关的房间号，switchCoord是开关在相应房间的坐标，activeState是一个状态值（例如在第四关中桥有三种状态，那么这个值就取0、1、2），当桥的状态值

与相应开关的状态值相等时才能安全通过。

物品：

宝箱中只有钥匙和剑这两种物品，盾牌是自带的。在 Lean 中，我们用一个枚举类型来建模物品：

```
inductive Item where
  | key
  | sword
  deriving DecidableEq, Repr
```

怪物：

由于游戏中的怪物攻击力和防御力都相同，也不存在任何正面或者负面增益效果，因此只需要记录怪物血量和位置就能表征一个怪物。在 Lean 中，我们用一个结构体来建模怪物：

```
structure Enemy where
  hp : Nat
  coord : Coord
```

我们对怪物进行了简化，即认为所有怪物都是chaser（始终向玩家移动），而不考虑更复杂的patroller和ambusher。

玩家：

玩家需要记录的信息有所处房间、在房间中的坐标、血量、金币数、钥匙数以及是否有剑。在 Lean 中，我们用一个结构体来建模玩家：

```
structure Player where
  room : Nat; coord : Coord; health : Nat
  gold : Nat; key : Nat; hasSword : Bool
```

条件：

在游戏中只存在三种条件（展示隐藏宝箱或者打开门的条件）：消耗钥匙、按下按钮或者击杀所有怪物。在 Lean 中，我们用一个枚举类型来建模条件：

```
inductive Condition where
  | None
  | consumeKey
  | ButtonPressed (buttonPos : Coord)
  | EnemyCleared
```

门：

对于门，需要记录门的编号，打开条件，是否打开，朝向，目标房间以及目标房间的目标坐标。在 Lean 中，我们用一个结构体来建模门：

```
structure DoorInfo where
  id : Nat
  condition : Condition
  isOpened : Bool
  orientation : Direction
  targetRoom : Nat
  targetCoord : Coord
```

房间：

对于一个房间，需要记录默认出生点来建模踩到陷阱的情况，还需要记录所有地块内容，所有门的信息，所有怪物信息，还有关于高度和宽度的不变量证明。在 Lean 中，我们用一个结构体来建模房间：

```
structure Room where
  spawn : Coord
  layout : List (List Tile)
  doors : List DoorInfo
  enemies : List Enemy
  inv_height : layout.length = 8
  inv_width : ∀ row ∈ layout, row.length = 10
```

游戏状态：

一个游戏状态由round（记录回合）、玩家信息和一个能由房间编号得到具体房间信息的函数组成。在 Lean 中，我们用一个结构体来建模游戏状态：

```
structure GameState where
  round : Nat
  player : Player
  rooms : Nat → Room
```

这里之所以需要回合是因为考虑到游戏中玩家速度是怪物两倍，因此我们规定怪物只能在偶数回合行动，也就大致模拟了速度倍数的关系。

状态转移（Env2.lean）

各种函数的作用：

Env2.lean 中的函数按功能分为四组，共同支撑最终的 step 归纳谓词。

辅助函数：

- `toFin8`：将自然数转化成 `Fin 8`
- `toFin10`：将自然数转化成 `Fin 10`

- `nth` : 查找列表第 `n` 个元素 获取游戏数据 :
- `front` : 得到当前位置指定方向上的下一个位置
- `getRoom` : 得到当前游戏状态下玩家所处房间
- `getTile` : 得到指定房间指定位置的地块
- `isChest` : 判断当前地块是否是宝箱
- `isSwitch` : 判断当前地块是否是开关
- `findEnemy` : 查找指定房间指定位置的怪物
- `findChest` : 查找指定房间指定位置的宝箱
- `findSwitch` : 查找指定房间指定位置的开关
- `findDoor` : 查找指定房间指定编号的门
- `moveAway` : 根据玩家位置和怪物位置判断怪物远离玩家的方向
- `moveTowardsPlayer` : 根据玩家位置和怪物位置判断怪物靠近玩家的方向
- `near` : 判断玩家和怪物是否相邻
- `isEnemyNear` : 判断怪物是否邻近玩家
- `conditionSatisfied` : 检查当前游戏状态下条件是否满足
- `orientationFit` : 判断玩家朝向和门的朝向是否符合

更新游戏数据 :

- `updateLayoutAt` : 以指定规则更新房间布局
- `updateLayoutAt_length` : 证明更新后房间长度满足要求
- `updateLayoutAt_width` : 证明更新后房间宽度满足要求
- `moveEnemy` : 将怪物向指定方向移动
- `killEnemy` : 玩家在一个状态下攻击面向的方向的怪物, 返回攻击后游戏状态
- `openChest` : 玩家在一个状态下打开指定位置的宝箱, 返回打开后游戏状态
- `toggleSwitch` : 玩家在指定房间指定位置切换开关, 返回切换后的房间信息
- `updatePlayer` : 玩家通过了指定的门之后玩家的信息
- `pushButton` : 玩家在指定房间指定位置按下按钮, 返回按下后的房间信息
- `updateEnemy` : 玩家所在房间怪物向玩家移动一步之后的房间信息
- `updateHealth` : 根据玩家是否开盾以及房间信息和玩家位置判断玩家扣血情况
- `bound` : 根据房间信息和玩家位置, 将和玩家相邻的怪物反弹

处理事件 :

- `handleWait` : 处理等待
- `handleMove` : 处理移动
- `handleInteract` : 处理交互
- `handleDefense` : 处理开盾

单步转移关系 :

根据现有状态和输入进行状态转移, 状态转移一步分为四种情况: 等待、移动、攻击或交互、防御。上述四种是玩家的动作, 而由于怪物只有移动这一种状态, 因此我们在每一步执行前判断是否是偶数回合, 若是则执行移动怪物、判断伤害、反弹造成伤害的怪物等等逻辑。在 Lean 中, 我们用一个归纳谓词来建模单步状态转移:

```
inductive step : GameState → Input → GameState → Prop where
  | wait      (s : GameState) (d : Direction) : step s {direction := d,
```

```

action := Action.wait} (handleWait { direction := d, action := Action.wait
} s)
| move      (s : GameState) (d : Direction) : step s {direction := d,
action := Action.move} (handleMove { direction := d, action := Action.move
} s d)
| interact (s : GameState) (d : Direction) : step s {direction := d,
action := Action.buttonA} (handleInteract { direction := d, action :=
Action.buttonA } s d)
| defense  (s : GameState) (d : Direction) : step s {direction := d,
action := Action.buttonB} (handleDefense { direction := d, action :=
Action.buttonB } s)

```

这是一个归纳谓词而非函数，每个构造子对应一种动作类型。step 使用 Prop 而非 Bool/GameState 返回，是因为我们希望在其上做归纳推理。

另外，updateLayoutAt 函数附带了两个关于布局更新的正确性定理，用来证明按照一定方式更新后长宽都不变，仍然符合房间约束。这两个定理被 openChest、toggleSwitch、pushButton 等函数内部使用，用于构造新的 Room 结构体时填充 inv_height 和 inv_width 证明字段。

2. 策略形式化

2.1 Strategy.lean 整体架构

Strategy.lean 对 Python 中的 student_agent.py 进行了符号层面的抽象建模。按功能分为 6 层：

```

Layer 1: opaque 环境查询接口
Layer 2: AgentState 结构体
Layer 3: Objective / ObjectiveKind 目标类型
Layer 4: chooseBridgeObjective — 中心室/拉杆房决策
Layer 5: chooseLocalObjective — 顶层目标选择
Layer 6: executeObjective / actLocalPlanner — 动作执行

```

Layer 1: opaque 环境查询

所有与环境交互的查询函数均被声明为 opaque（不可展开），其行为仅通过公理约束：

opaque 函数	对应 Python 行为
getCurrentMonsters	从感知输出中提取怪物坐标列表
getCurrentChests	从感知输出中提取宝箱坐标列表
getCurrentExits	从感知输出中提取出口坐标列表
getCurrentSwitches	从感知输出中提取开关坐标列表
isAdjacent	判断两坐标是否相邻（曼哈顿距离 = 1）
getDirectionTo	给定相邻坐标，返回朝向方向
nextStepTo	BFS 寻路：从 start 到 goals（避开 blocked），返回下一步方向

opaque 函数	对应 Python 行为
<code>hasBridge / getBridgeSides</code>	桥/中心室特征判定
<code>sideApproachGoals</code>	计算从某侧接近出口的 approach tile 列表

为什么使用 opaque？(1) 这些函数在 Python 中由 BFS/A* 搜索或 CNN 感知实现，展开到 Lean 中将导致证明过于冗长且与具体搜索算法耦合；(2) opaque + 公理的方式将对具体实现的依赖转化为对行为规约的依赖，使得证明可以适配不同搜索策略或感知模型。

Layer 2: AgentState

```
structure AgentState where
  hubExplorationStarted      : Bool
  rememberedBlockers         : List Coord    -- 已交互/应屏蔽的坐标（开过的宝箱、杀
过的怪物等）
  monsterObjectiveDone       : Bool
  sawMonsterObjective        : Bool
  monsterAbsenceTicks        : Nat
  currentTargetSide          : Option Direction
  switchHubSide              : Option Direction
  hubSwitchPositions         : List Coord
  pressedSwitchForTarget     : Bool
  exploredHubSides           : List Direction
  postGoalRotateBridge       : Bool
  postGoalSwitchPressed      : Bool
```

关键字段说明：`rememberedBlockers` 是安全性/活性证明的核心——它将已交互对象记录为“屏蔽坐标”，寻路时避开。`blockersCoverObstacles` 谓词断言所有墙和陷阱均在 blockers 中，这是定理 1~4 的前提条件。

Layer 3~4: 目标选择

```
inductive ObjectiveKind where
| fight      -- 攻击怪物
| interact   -- 交互（开宝箱/按按钮/切换开关）
| navigate    -- 导航到目标坐标集
| goExit     -- 前往某侧出口
| idle       -- 空闲

structure Objective where
  kind : ObjectiveKind
  targets : List Coord
  side : Option Direction
  interactionKind : Option InteractionKind
```

`chooseLocalObjective` 的优先级逻辑（对应 Python 中的 `_choose_local_objective`）：

1. 有怪物且 (有剑 或 非 Hub 探索模式) → `fight`
2. 有有效宝箱 (不在 blockers 中) → `interact`
3. Hub 探索模式 → `chooseBridgeObjective` (处理桥/拉杆房逻辑)
4. 否则 → `goExit` (有可见出口侧) 或 `navigate` (向出口坐标集导航)

`t1~t4` 的 `chooseLocalObjective` 调用中 `useHubExploration = false`, 因此 `AgentState` 在执行 `executeObjective` 后保持不变。这是定理 3&4 能够将 `AgentState` 排除在归纳测度之外的关键前提。

Layer 5~6: 动作执行

`executeObjective` 将 `Objective` 翻译为 `Input × AgentState`。逻辑分支：

ObjectiveKind	玩家邻近目标？	输出
<code>fight</code>	是	<code>getDirectionTo</code> → <code>ACTION_A</code> (攻击)
<code>fight</code>	否	<code>nextStepTo</code> → <code>ACTION_MOVE</code>
<code>interact</code>	是	<code>getDirectionTo</code> → <code>ACTION_A</code> (交互)
<code>interact</code>	否	<code>nextStepTo</code> → <code>ACTION_MOVE</code>
<code>goExit</code>	在 approach tile 上	直接向 <code>side</code> 方向移动
<code>goExit</code>	不在 approach tile 上	<code>nextStepTo</code> 到 approach tile
<code>navigate</code>	—	<code>nextStepTo</code> → <code>ACTION_MOVE</code>
<code>idle</code>	—	<code>WAIT</code>

2.2 关键设计选择

选择 1 : `t1~t4` 统一策略 vs 每关单独

我们用单个 `strategyTasks1to4` 函数覆盖全部前四关，而非为每关写独立策略。这体现了“统一符号规划器”的思路，保证了策略的结构通用性。

选择 2 : `AgentState` 不变性

```
def strategyTasks1to4 (gs : GameState) (as : AgentState) : Input ×
AgentState :=
  let (obj, as') := chooseLocalObjective gs as false
  executeObjective gs as' obj
```

通过 `chooseLocalObj_snd_eq` 和 `executeObjective_snd_eq` 公理，可证明：

```
theorem strategy_snd_eq (gs : GameState) (as : AgentState) :
  (strategyTasks1to4 gs as).snd = as := ...
```

推论：在 $t_1 \sim t_4$ 中，`AgentState` 在每一步执行后与执行前完全相同。这直接简化了 `steps` 关系的定义（只需一个 `as` 参数在每一步重复使用）和活性证明的归纳假设（只需对 `GameState` 做归纳，`as` 作为常量）。

选择 3：steps 多步执行关系

```
inductive steps : GameState → AgentState → Nat → GameState → AgentState → Prop where
| refl (gs : GameState) (as : AgentState) : steps gs as 0 gs as
| succ (gs : GameState) (as : AgentState) (gsMid : GameState)
  (gs' : GameState) (as' : AgentState) (n : Nat) :
  step gs (strategyTasks1to4 gs as).fst gsMid →
  steps gsMid as n gs' as' →
  steps gs as (n + 1) gs' as'
```

与标准的 reflexive-transitive closure 不同，这里：

- 每一步都使用 `strategyTasks1to4` 决策（而非任意的 `Input`），因为我们要证明的是“策略+环境闭环系统”的性质，而非环境本身的任意迹
- `succ` 构造子中 `steps gsMid as n gs' as'` 的 `as` 不变——利用策略不修改 `AgentState` 的性质
- 步数 `n` 是显式的自然数参数，使强归纳法能够按步数/测度进行

3. 定理与证明

我们没有针对 task 5 的 `global_planner` 进行性质证明，而是证明了 `local_planner` 的以下性质。

3.1 定理概览

定理	类型	直观含义	形式陈述
定理 1 <code>no_wall_bump</code>	安全性 (Safety)	策略输出 move 时， 前方不是墙	<code>getTile ... (front ...) ≠ Tile.wall</code>
定理 2 <code>no_spike_step</code>	安全性 (Safety)	策略输出 move 时， 前方不是陷阱	<code>getTile ... (front ...) ≠ Tile.spike</code>
定理 3 <code>single_monster_kill</code>	活性 (Liveness)	单怪物场景下，有限 步内怪物必被消灭	$\exists n \text{ gs' as' }, \text{steps gs0 as0 n gs' as' } \wedge \text{enemies} = []$
定理 4 <code>single_chest_open</code>	活性 (Liveness)	无怪物、有有效宝箱 场景下，有限步内宝 箱全开	$\exists n \text{ gs' as' }, \text{steps ... } \wedge \text{validChests} = []$

3.2 定理 1 & 2：安全性证明

前提条件

两个定理共享同一个前置条件 `blockersCoverObstacles gs as.rememberedBlockers`，即：

```
def blockersCoverObstacles (gs : GameState) (blockers : List Coord) : Prop
:=
  ∀ (c : Coord),
    (let t := getTile (getRoom gs) c; t = Tile.wall ∨ t = Tile.spike) →
      c ∈ blockers
```

这意味着 rememberedBlockers 包含了所有墙和陷阱坐标。结合 nextStepTo_avoids_blocked 公理——"nextStepTo 返回的方向不会走向 blockers 中的坐标"——即可推出安全性。

证明流程

对 Objective.kind 进行五项分类讨论：

Step 1：展开策略到 executeObjective 调用层

```
rw [strategy_expand gs as] at hMove ⊢
generalize hPair : chooseLocalObjective gs as false = pair
rcases pair with (obj, as')
```

Step 2：证明 as'.rememberedBlockers = as.rememberedBlockers（利用 chooseLocalObj_snd_eq 公理），将 hCovers 迁移到 hCovers'：blockersCoverObstacles gs as'.rememberedBlockers。

Step 3：对 obj.kind 分类：

obj.kind	action = move 何时出现？	安全论证
idle	永不出 move	前提矛盾，自动消去
fight	不邻近任何怪物 → nextStepTo 返回移动方向	nextStepTo_avoids_blocked 保证目标格 ∉ blockers，而 blockers 包含所有墙/陷阱
interact	不邻近任何交互对象 → nextStepTo 返回移动方向	同上
goExit	在 approach tile 上 → 向出口方向移动；否则 → nextStepTo 到 approach tile	前者用 exit_direction_not_wall/exit_direction_not_spike；后者同上
navigate	nextStepTo 返回移动方向	nextStepTo_avoids_blocked

定理 1（不撞墙）：

- 在 `nextStepTo_avoids_blocked` 分支中：`hAvoid : ¬ (front start d ∈ blocked)`，引入 `hWall : ... = Tile.wall`，则 `apply hAvoid; apply hCovers'; left; exact hWall` → 矛盾
- 在 `goExit` 的 `approach` 分支中：直接调用 `exit_direction_not_wall`

定理 2 (不踩陷阱)：

- 结构完全对称，区别仅在于 `nextStepTo_avoids_blocked` 分支用 `hCovers'; right; exact hSpike`
- 在 `goExit` 的 `approach` 分支中：调用 `exit_direction_not_spike`
- `fight` 分支使用 `generalize hFilter` 同步 `head?` 结果，避免 Lean 将 `head?` 展开导致无法匹配

定理 2 的特殊技巧：`generalize`

`fight` 分支中 `executeObjective` 内部有 `(obj.targets.filter (fun m => isAdjacent ...)).head?`。如果直接在 `hMove` 中展开，后续 `match` 分支的类型会依赖于具体的 `head?` 结果 (`some m / none`)，且不同分支在 Lean 眼中是不同类型，无法统一处理。因此使用：

```
generalize hFilter : (obj.targets.filter (fun m => isAdjacent
gs.player.coord m)).head? = filterResult
```

将 `head?` 结果抽象为变量 `filterResult`，在 `filterResult` 的不同值上分别处理，保持各分支类型一致。

3.3 定理 3 & 4：活性证明

3.3.1 总体架构：强归纳法 + 终止测度

两个活性定理都采用良基归纳法 (well-founded induction) ——具体是自然数上的强归纳法 `Nat.strongRecOn`：

目标：证明对所有 `GameState`，存在有限步 `n` 达成目标状态
 方法：对“终止测度” `m` 做强归纳

- 归纳假设：对所有测度 `< m` 的状态，性质成立
- 归纳步骤：证明对测度 `= m` 的状态，经一步后测度严格减小（由核心公理保证）
 → 落到归纳假设覆盖范围 → 存在有限步

这与编程语言中证明程序终止的典型方法一致：构造一个严格递减且下有界的测度函数。

3.3.2 测度设计

定理 3 (怪物击杀) 的测度：

```
def monsterMeasure (gs : GameState) : Nat :=
  totalMonsterHP gs * 100 + minCoordDist gs.player.coord
```

```
(getCurrentMonsters gs)
```

分量	含义	变化趋势
<code>totalMonsterHP gs</code>	当前房间所有怪物 HP 之和	攻击命中时至少 -1
<code>minCoordDist ...</code>	玩家到最近怪物的曼哈顿距离	向怪物移动时缩小

测度设计原理：乘数 **100** 是关键——它远大于网格上任意两点之间的曼哈顿距离（ 8×10 网格中最大距离 = $7 + 9 = 16$ ）。这保证了测度在**字典序**上的行为：

- 如果 HP 减少 $\rightarrow$ 测度减少 ≥ 100 ，无论距离如何变化
- 如果 HP 不变 $\rightarrow$ 测度完全由距离决定，BFS 移动每步使距离严格减小

这使得测度在每一步要么因 HP 减少而大幅下降，要么因靠近怪物而小幅下降——二者结合保证严格单调递减。

定理 4 (宝箱打开) 的测度：

```
def chestMeasure (gs : GameState) (blockers : List Coord) : Nat :=
  let validChests := (getCurrentChests gs).filter (fun c =>
    !blockers.contains c)
  validChests.length * 100 + minCoordDist gs.player.coord validChests
```

分量	含义	变化趋势
<code>validChests.length</code>	未被 blocker 屏蔽的宝箱数	开箱成功时 -1
<code>minCoordDist ...</code>	玩家到最近有效宝箱的距离	向宝箱移动时缩小

与定理 3 同样的乘数 100 设计——打开宝箱使有效宝箱数 -1，测度减少 ≥ 100 ，主导距离的任何可能波动。

3.3.3 定理 3 的详细证明流程

前置条件：

```
theorem single_monster_kill (gs0 : GameState) (as0 : AgentState)
  (hMonsterCount : (getCurrentMonsters gs0).length = 1) -- 恰好一只怪物
  (hHasSword : gs0.player.hasSword = true) -- 玩家有剑
  (hCover : blockersCoverObstacles gs0 as0.rememberedBlockers) : --
  blockers 覆盖障碍物
   $\exists$  (n : Nat) (gs' : GameState) (as' : AgentState),
    steps gs0 as0 n gs' as'  $\wedge$  (getRoom gs').enemies = []
```

证明步骤：

(a) 推导 `isEmpty = false`：从 `length = 1  $\rightarrow$  getCurrentMonsters gs0  $\neq$  []  $\rightarrow$  isEmpty = false`。

(b) 定义归纳性质 P :

```

let P : Nat → Prop := λ n => ∀ (gs : GameState) (as : AgentState),
  monsterMeasure gs = n → -- 测度 = n
  (getCurrentMonsters gs).isEmpty = false → -- 有怪物
  gs.player.hasSword = true → -- 有剑
  blockersCoverObstacles gs as.rememberedBlockers → -- 障碍覆盖
  ∃ (n' : Nat) (gs' : GameState) (as' : AgentState),
    steps gs as n' gs' as' ∧ (getRoom gs').enemies = []

```

P n 说的是：对所有测度等于 n 且满足前置条件的状态，存在有限步使怪物被消灭。

(c) 归纳步骤 hStep : $\forall n, (\forall m < n, P m) \rightarrow P n$:

1. 判断 enemies 是否已空：若 enemies = [], 返回 $n' = 0$ (零步, steps.refl)
2. 从核心公理获取一步进展：

```

have hProgress : ∃ gs', step gs (strategyTasks1to4 gs as).fst gs' ∧
  monsterMeasure gs' < monsterMeasure gs :=
  not_not_elim (strategy_fight_step_decreases_measure gs as
    hMonstersNE hSword hCoverArg)

```

得到中间状态 gsMid、转移 hStepMid、测度减小 hMeasureLt。

3. 重建归纳假设的前置条件：

- hSwordMid : 由 hasSword_preserved_step 公理保证 gsMid.player.hasSword = true
- hCoverMid : 由 blockersCoverObstacles_preserved 公理保证 blockers 覆盖在 gsMid 中仍成立

4. 判断 gsMid 中怪物状态：

- 仍有怪物 : hm : monsterMeasure gsMid < n (由 hMeasureLt 和 hMeasure)，使用归纳假设 hn (monsterMeasure gsMid) hm gsMid as rfl hMonstersNEMid hSwordMid hCoverMid，得到 n' 步后的结果，加上当前这一步 → 返回 n' + 1
- 怪物已消灭 : 返回 n' = 1 (一步达成)

(d) 启动归纳：

```

have hAll : ∀ n, P n := λ n => Nat.strongRecOn n hStep
exact hAll (monsterMeasure gs0) gs0 as0 rfl hMonstersNE0 hHasSword hCover

```

3.3.4 定理 4 的详细证明流程

定理 4 的整体结构与定理 3 类似，但有一些值得报告的差异：

差异 1：归纳性质仅参数化 GameState

```
let blockers := as0.rememberedBlockers -- 常量! 策略不修改 as
let P : Nat → Prop := λ n => ∀ (gs : GameState),
  chestMeasure gs blockers = n →
  (getCurrentMonsters gs).isEmpty = true →
  gs.player.key = 0 →
  blockersCoverObstacles gs blockers →
  (getCurrentChests gs).filter (fun c => !blockers.contains c) ≠ [] →
  ∃ (n' : Nat) (gs' : GameState),
    steps gs as0 n' gs' as0 ∧ -- as0 直接出现在结论中
    (getCurrentChests gs').filter (fun c => !blockers.contains c) = []
```

因为 t1~t4 中策略不修改 `AgentState`，`as0` 自始至终不变。`blockers` 定义为 `as0.rememberedBlockers`，在全部归纳步骤中保持常量。

差异 2：目标条件是有效宝箱集为空

定理 4 的目标不是"所有宝箱都是 opened 状态"，而是"有效宝箱集（不在 blockers 中的未开宝箱）为空"。这与 `chestMeasure` 的定义一致：`validChests.length * 100 + ...`。

差异 3：需要更多不变量的保持

除了 `blockersCoverObstacles_preserved` 之外，定理 4 的归纳步骤还需：

- `noMonsters_preserved_step`：无怪物状态在 step 中保持（怪物不会凭空产生）
- `key_stays_zero_while_chests`：有有效宝箱等待开启时，key 保持为 0（宝箱内容不产生钥匙，或系统的钥匙不会改变有效宝箱集）

`key_stays_zero_while_chests` 公理是定理 4 的一个关键简化假设——它排除了"宝箱含钥匙 → key > 0 → 行为改变"的复杂情况。

差异 4：最终结果组装

因为 `P n` 返回 `∃ gs', steps gs as0 n' gs' as0 ∧ ...`（只量化了 `gs'`，不含 `as'`），而定理陈述要求 `∃ n gs' as', ...`，最终用 `rcases + exact (n, gs', as0, hSteps, hAllOpened)` 组装。

3.3.5 not_not_elim 辅助引理

由于 Lean 4.29-rc6 解析器 bug，所有公理的返回类型包装在 `¬ (¬ ...)` 中。证明中需要用 `not_not_elim` 提取正向陈述：

```
private theorem not_not_elim {P : Prop} (h : ¬ (¬ P)) : P := by
  by_cases hP : P
  · exact hP
  · exfalso; exact h hP
```

使用方式：

```
have hPreservedWrapped := hasSword_preserved_step gs gsMid ... hStepMid
have hPreserved : gsMid.player.hasSword = gs.player.hasSword :=
  not_not_elim hPreservedWrapped
```

这是一种**工程折中**——理想情况下公理应直接陈述正向命题，但受限于工具链的解析器 bug 而采用双重否定包装。

4. 公理体系

4.1 公理分类与完整清单

共享辅助定义（4 个，非公理）

定义	签名	用途
coordDist	Coord → Coord → Nat	曼哈顿距离
minCoordDist	Coord → List Coord → Nat	到坐标集的最小曼哈顿距离
totalMonsterHP	GameState → Nat	当前房间所有怪物 HP 之和
blockersCoverObstacles	GameState → List Coord → Prop	blockers 是否包含所有墙和陷阱

安全性公理（5 条，定理 1 & 2）

公理	完整签名	角色
nextStepTo_avoids_blocked	nextStepTo start goals blocked = some d → ¬ (front start d ∈ blocked)	寻路不走向 blocked 坐标 — 安全性的直接来源
exit_direction_not_wall	sideApproachGoals gs side blocked 包含 pCoord → ¬ (getTile ... (front pCoord side) = Tile.wall)	出口 approach tile 向出口方向移动时不撞墙
exit_direction_not_spike	同上，结论为 ≠ Tile.spike	同上，不踩陷阱
getDirectionTo_correct	getDirectionTo src dst = some d → ¬ (¬ (front src d = dst))	方向计算正确性 — 面向目标
isAdjacent_correct	¬ (¬ (isAdjacent c1 c2 = true ↔ near c1 c2))	isAdjacent 与 near（曼哈顿距离 =1）等价

活性公理（17 条，定理 3 & 4）

寻路相关（3 条）：

公理	含义
<code>nextStepTo_some</code>	目标集非空且不邻近任何目标时， <code>nextStepTo</code> 总能返回方向（BFS 完备性）
<code>nextStepTo_progress</code>	<code>nextStepTo</code> 返回的方向使玩家到目标集的最小曼哈顿距离严格减小（BFS 正确性）
<code>getDirectionTo_adjacent</code>	两坐标相邻时， <code>getDirectionTo</code> 总能返回方向

环境状态查询（2 条）：

公理	含义
<code>getCurrentMonsters_empty_iff</code>	<code>getCurrentMonsters</code> 为空 ↔ 房间 <code>enemies</code> 为空
<code>getCurrentMonsters_mem</code>	坐标在 <code>getCurrentMonsters</code> 中 ↔ 该坐标有怪物

状态转移语义（3 条）：

公理	含义
<code>killEnemy_totalHP_lt</code>	有剑且前方有怪物时， <code>killEnemy</code> 使总 HP 严格减小
<code>handleMove_player_coord</code>	前方非墙非刺时，移动后玩家坐标 = 前方坐标
<code>handleMove_totalHP_nonincrease</code>	移动操作不增加怪物总 HP（怪物不会在移动中回血/产生）

宝箱语义（2 条）：

公理	含义
<code>openChest_opens</code>	<code>openChest</code> 将未开启宝箱 (<code>chest false ...</code>) 变为已开启 (<code>chest true ...</code>)
<code>getCurrentChests_excludes_opened</code>	<code>getCurrentChests</code> 排除已开启 (<code>chest true ...</code>) 的宝箱

状态不变性（5 条）：

公理	保持什么	为什么需要
<code>hasSword_preserved_step</code>	<code>hasSword</code>	定理 3 归纳步骤需保证有剑状态不会丢失
<code>blockersCoverObstacles_preserved</code>	<code>blockersCoverObstacles</code>	定理 1~4 归纳步骤均需保证障碍覆盖性质不变
<code>noMonsters_preserved_step</code>	<code>noMonsters</code>	定理 4 归纳步骤需保证在宝箱场景中不会出现新怪物
<code>key_stays_zero_while_chests</code>	<code>key = 0</code>	定理 4 归纳步骤需保证 key 不变成正数（避免触发新

公理	保持什么	为什么需要
		行为分支)
<code>executeObjective_snd_eq</code>	<code>AgentState</code>	定理 1&2 <code>hAs'</code> 推导 + 定理 3&4 <code>as</code> 不变性

核心活性公理（2 条）：

公理	完整含义
<code>strategy_fight_step_decreases_measure</code>	有怪物 + 有剑 + 障碍覆盖 → 存在一步转移使 <code>monsterMeasure</code> 严格减小
<code>strategy_interact_step_progress</code>	无怪物 + 有有效宝箱 + <code>key=0</code> + 障碍覆盖 → 存在一步转移使 <code>chestMeasure</code> 严格减小

这两条是最关键的活性公理，封装了"策略+环境"闭环的单步进展保证。它们综合了寻路进展（`nextStepTo_progress`）、攻击效果（`killEnemy_totalHP_lt`）、开箱效果（`openChest_opens`）、移动正确性（`handleMove_player_coord`）等底层公理的组合效应。

4.2 公理的合理性论证框架

课程要求"不允许未说明的 `axiom`"。下面从四个角度论证每条公理的合理性：

角度 1：对应 Python 实现

每条公理可以关联到 Python 代码中的具体行为：

公理	Python 对应
<code>nextStepTo_avoids_blocked</code>	<code>_bfs</code> 返回的路径第一步不会走向 <code>blocked</code> 参数中标记的坐标
<code>nextStepTo_progress</code>	BFS 每步扩展使 <code>frontier</code> 到目标的距离缩短
<code>nextStepTo_some</code>	只要目标集非空且玩家不在目标上，BFS 总能找到一条路径（网格连通前提）
<code>killEnemy_totalHP_lt</code>	<code>killEnemy</code> 中 <code>hp - 1</code> 或 <code>filter (en => en.coord != coord)</code> 使 HP 总和减少
<code>handleMove_player_coord</code>	<code>handleMove</code> 的 <code>Tile.ground / Tile.switch / Tile.door</code> 分支设置 <code>coord := front ...</code>
<code>openChest_opens</code>	<code>openChest</code> 中 <code>updateLayoutAt</code> 将 <code>Tile.chest false ...</code> 替换为 <code>Tile.chest true ...</code>

角度 2：环境语义保证

有些公理描述的是环境本身的不变性质，不依赖策略：

- `handleMove_totalHP_nonincrease`：移动操作不会创造怪物或给怪物回血，这是模拟器语义的内在性质

- `noMonsters_preserved_step` : 怪物不会凭空产生（只在房间初始化时存在），这也是环境的语义保证
- `hasSword_preserved_step` / `blockersCoverObstacles_preserved` : 这些是环境状态的自然不变量

角度 3 : 策略行为保证

有些公理描述策略内部调用的函数的正确性——这些函数在 Python 中有确定的实现，但 Lean 中因声明为 `opaque` 而需要公理：

- `executeObjective_snd_eq` : 从 `executeObjective` 的代码可见，每个分支返回的 `as` 都是传入的 `as ({direction := ..., action := ...}, as)`，从未修改
- `getDirectionTo_adjacent` : `getDirectionTo` 在两坐标相邻时只需比较坐标差即可确定方向

角度 4 : 可独立验证

每条公理描述的行为都可以在 Python 环境中通过单元测试验证：

- 设置特定 `GameState` → 调用对应函数 → 检查返回值或状态变化是否符合公理断言
- 这将形式化证明与“代码确实运行”的可信性连接起来

4.3 公理与定理的依赖关系

```

安全性公理 (nextStepTo_avoids_blocked,
             exit_direction_not_wall/spike,
             getDirectionTo_correct)
  ↳ 定理1 (no_wall_bump)
  ↳ 定理2 (no_spike_step)

底层寻路公理 (nextStepTo_some, nextStepTo_progress, getDirectionTo_adjacent)
底层状态公理 (killEnemy_totalHP_lt, handleMove_player_coord,
             handleMove_totalHP_nonincrease, openChest_opens, ...)
  ↳ 核心活性公理 (strategy_fight_step_decreases_measure,
                 strategy_interact_step_progress)
    ↳ 定理3 (single_monster_kill) ← + hasSword_preserved_step,
                                   |
blockersCoverObstacles_preserved
    ↳ 定理4 (single_chest_open) ← + noMonsters_preserved_step,

key_stays_zero_while_chests

```

5. 形式化范围与局限性

已覆盖

- `Coord`、`Direction`、`Action`、`Tile`、`Enemy`、`Player`、`Room`、`GameState` 等核心类型的完整建模
- 四种动作 (`move` / `interact` / `wait` / `defense`) 的状态转移关系 (`step` 归纳谓词)

- 基于 `blockers` 的障碍物屏蔽机制（将已开宝箱/已杀怪物等从寻路目标中排除）
- 安全性：策略输出 `move` 时前方如果是墙/陷阱则与公理矛盾 → 一定不是墙/陷阱
- 活性：单怪物 + 有剑 → 有限步消灭；无怪物 + 有宝箱 + `key=0` → 有限步全开

未覆盖 / 有意的简化

简化项	当前假设	实际 Python 环境	影响
多怪物场景	定理 3 假设 <code>length = 1</code>	t4/t5 中存在多怪物	多怪物收敛性需要更复杂的测度（如 $\sum HP_i \times 100 + \text{minDist}_i$ 的字典序）
宝箱内容多样性	定理 4 假设宝箱不产生钥匙（ <code>key_stays_zero_while_chests</code> ）	t3/t4 中钥匙宝箱 → <code>key > 0</code> 后行为改变	需要分阶段证明：先拿钥匙 → 用钥匙开门 → 再处理其他宝箱
跨房间导航	定理 3&4 仅涉及单房间内的性质	t3~t5 涉及多房间移动	<code>room_memory / exit_memory</code> / 房间签名匹配等判定逻辑未被形式化
桥/按钮/开关机制	未形式化	t4/t5 的核心机制	<code>chooseBridgeObjective</code> 的复杂状态机逻辑未被证明
怪物 AI 行为	怪物同质化（不区分 <code>chaser/patroller/ambusher</code> ）	Python 中三种 AI 有所不同	不区分 AI 不影响安全性证明（因为怪物移动不改变墙/陷阱布局），但可能影响活性测度的单调性
像素级移动	Lean 中 <code>Action.move</code> = tile 级移动（一次一格）	Python 中是像素级（16 帧同方向 = 1 tile）	有意的抽象层简化——tile 级建模足以刻画所有安全性和目标可达性
感知不确定性	Lean 中 <code>GameState</code> 完全已知、可直接访问	Python 策略只能通过 <code>obs</code> 图像帧推断	Lean 证明"给定完美感知"下的性质；Python 实际表现另需感知模块准确性保障

多层验证架构

本次 Lean 形式化验证在整个验证体系中的定位如下：



obs(像素) → PerceptionEngine → 符号状态 → choose_objective → BFS/A* → action 验证方式：鲁棒性测评 (--robustness-suite)	
Lean 层：符号逻辑验证 GameState(完美已知) → strategyTasks1to4 → step → GameState' 验证方式：安全性定理 + 活性定理 前提：感知模块输出正确 (准确提取符号状态)	

两层验证之间的**信任鸿沟**是感知准确性：Lean 证明的策略安全性和活性是在"感知输出完全正确"的前提下成立的。Python 端通过鲁棒性测评（颜色变体 + 空间变体测试）来建立对感知模块实际准确性的信心。